

大数据存储技术作业说明

本作业提供两种形式, 同学可根据自身兴趣和技术水平任选其一完成。

1. 调研报告 (总分 100 分) 难度***
2. 分布式数据库实现 (总分 100+20 分) 难度****

一、 调研报告

1. 作业目标

- 系统梳理大数据存储技术的发展脉络与演进逻辑
- 深入分析主流/前沿存储技术的核心架构与设计权衡
- 培养文献检索、技术对比、学术写作能力

2. 调研主题

请从以下四个方向中任选其一, 撰写一篇 5000-8000 字的调研报告。

1) 方向 1: 分布式文件系统的演进之路

- 核心问题:
 - 从 GFS 到 HDFS, 再到 JuiceFS、Alluxio, 架构演进解决了什么问题?
 - 元数据管理方案的演进: 单点 Master → 分布式元数据 → 元数据分离
 - 计算与存储分离架构的优劣势
- 分析系统: GFS、HDFS、JuiceFS、Alluxio、Ceph FS (至少 3 个)

2) 方向 2: NoSQL 数据库的技术分化与融合

- 核心问题:

- 为什么 NoSQL 会分化出 KV、文档、列族、图等不同类型?
- 各类 NoSQL 的核心数据模型与索引结构差异。
- 近年来 NoSQL 与 SQL 的边界正在模糊 (NewSQL、Multi-Model)。

- 分析系统: (选 3-4 个对比)

- KV: Redis、RocksDB
- 列族: HBase、Cassandra
- 文档: MongoDB
- 多模型: ArangoDB、OrientDB

3) 方向 3: 分布式一致性协议的工业化实践

- 核心问题:

- Paxos 与 Raft 的核心差异: 为什么 Raft 更容易工程化?
- Raft 在典型系统中的应用与改造 (Multi-Raft、Parallel Raft)
- 强一致性与可用性的权衡: CAP 理论的实际落地

- 分析内容:

- Paxos 与 Raft 协议对比
- Raft 在 etcd、TiKV、CockroachDB 中的工程实现差异
- Lease read、Follower read 等优化手段

4) 方向 4：云原生时代的存储架构

- 核心问题：
 - 云存储的三大形态：块存储、文件存储、对象存储
 - 存算分离架构的挑战：网络延迟、数据缓存、元数据管理
 - Serverless 数据库：Aurora、Snowflake、TiDB Cloud
- 分析系统：AWS S3、Amazon Aurora、Snowflake、TiDB

3. 报告结构要求

1. 引言

- 背景与意义
- 报告范围与研究方法

2. 技术发展脉络

- 时间线梳理（附技术演进图）
- 驱动因素分析（数据规模、硬件发展、业务需求）

3. 核心系统对比分析

- 每个系统的架构图（必须自己绘制）

- 设计哲学与适用场景
- 关键设计决策分析（至少 3 个关键技术点）
- 4. 技术权衡讨论
 - 一致性 vs 可用性 vs 分区容错
 - 读优化 vs 写优化
 - 通用性 vs 专用性
- 5. 未来趋势展望
 - 至少 3 个可预见的演进方向
 - 附支撑性参考文献
- 6. 总结
- 7. 参考文献（不少于 15 篇，含顶会论文）

4. 评分标准

维度	占比	具体要求
技术深度	30%	对核心协议、算法、架构的准确理解
对比分析	25%	多系统横向对比，有量化指标或定性分析
原创性	20%	有自己的见解，非简单资料堆砌
图表质量	10%	自绘架构图、时序图、对比表
文献质量	10%	引用顶会（SOSP/OSDI/SIGMOD）论文
写作规范	5%	逻辑清晰，术语准确，引用规范

二、 分布式数据库实现

1. 作业名称：基于 Raft 的分布式 KV 数据库实现

2. 作业要求:

实现一个基于 Raft 协议的分布式 KV 数据库, 要求:

- 1) 支持多节点集群 (至少 3 节点)
- 2) 任一节点宕机, 系统仍可用 (容忍 1 台故障)
- 3) 提供简单的 Put/Get/Delete 接口

3. 核心功能模块

1) 模块 1: Raft 核心协议 (40)

- Leader 选举随机超时、心跳维持、任期管理
- 日志复制 AppendEntries RPC, 日志匹配检查
- 提交机制 Leader 维护 committedIndex, 通知 Follower

2) 模块 2: KV 状态机 (25)

- Put(key, value): 通过 Raft 同步写入
- Get(key) 从 Leader 读取 (线性一致读需优化)
- Delete(key) 通过 Raft 同步删除

3) 模块 3: 网络通信 (15)

- RPC 框架 实现 RequestVote 和 AppendEntries
- 客户端接口 HTTP 或 TCP 接口供调用

4) 模块 4: 故障恢复 (20)

- 节点重启恢复 从持久化状态恢复并追赶日志
- Leader 切换 Leader 宕机后自动选举新 Leader

5) 加分功能 (+20)

- 快照机制 日志压缩, 防止日志无限增长
- 成员变更 支持动态添加/移除节点
- 线性一致读优化 ReadIndex 或 Lease read, 不提交

日志

4. 交付物

- 1) 开发报告文档
- 2) 软件代码

5. 文档要求

1. 系统架构设计

- 模块划分与职责
- 核心数据结构定义

2. Raft 协议实现细节

- Leader 选举流程图
- 日志复制时序图
- 持久化内容说明

3. 关键问题与解决方案

- 网络分区处理
- 日志冲突解决
- 选票分裂预防

4. 测试方案与结果

- 单节点宕机恢复测试
- 多节点故障测试

- 脑裂场景测试（需要验证不会出现双 Leader）

5. 未成功能与改进方向

6. 评分标准

维度	占比	具体要求
Leader 选举	25%	随机超时、任期递增、多数投票
日志复制	15%	日志匹配、提交推进、一致性检查
通信	10%	RPC 框架，客户端接口
KV 状态机	20%	Put/Get/Delete 正确性
故障容错	20%	任 1 节点宕机仍可用，
测试覆盖	10%	测试脚本、边界场景
加分项	+20	快照、成员变更、线性一致读